



Guia do Aluno — Lab da Aula 4

Pipeline Olist: GitHub → Airflow → Snowflake

Data Integration e Pipelines · FIAP MBA Data Engineering

Prof. Rafael S Novo Pereira

Apache Airflow 3.3.2 (Astro Runtime 3.3-9) · Astronomer Astro · Snowflake · Kubernetes · validado de ponta a ponta em 08/10/2026

Como usar este guia

Siga as etapas na ordem. Cada uma termina com um ponto de verificação  — só avance quando ele estiver OK.

Os quadros  mostram o erro mais comum daquela etapa e como resolver antes de levantar a mão.

Parte 1 (Etapas 1–16): o pipeline rodando — é o desafio Bronze. Parte 2 (Etapas 17–18): duas experiências que valem mais do que parecem. Parte 3: desafios Prata e Ouro.

Tempo: 60–90 min para a Parte 1. Trabalhe em dupla (um Astro por dupla), mas a entrega é individual.

Antes de começar (10 min, em casa ou no início da aula)	4
0.1 — Snowflake ativo	4
0.2 — Conta no GitHub e fork do repositório	4
0.3 — Regras dos trials	4
Parte 1 — O pipeline rodando (Etapas 1 a 16 · 60–90 min)	5
Etapa 1 — Criar conta no Astro	5
Etapa 2 — Selecionar o uso	5
Etapa 3 — Configurar Organization e Workspace	6
Etapa 4 — Pular o template inicial	6
Etapa 5 — Tela principal do Astro	7
Etapa 6 — Configurar o Deployment	7
Etapa 7 — Aguardar o Deployment ficar HEALTHY	9
Etapa 8 — Verificar o Airflow	10
Etapa 9 — O código no GitHub	10
Etapa 10 — Conectar o GitHub ao Astro IDE	11
Etapa 11 — Deploy do projeto	12
Etapa 12 — Verificar as DAGs no Airflow	14
Etapa 13 — Configurar a Connection com Snowflake	16
Etapa 14 — Executar o pipeline	23
Etapa 15 — Pipeline completo com sucesso	24
Etapa 16 — Verificar os resultados no Snowflake	24
Parte 2 — Duas experiências que valem mais do que parecem (15 min)	27
Etapa 17 — Backfill: a task não sabe "hoje"	27
Etapa 18 — Idempotência: rodar duas vezes é seguro?	27
Parte 3 — Desafios (entrega individual pelo portal)	29
🥈 Prata A — Faça um quality check falhar de verdade	29
🥈 Prata B — Dê um calendário à DAG e faça backfill	29
🥈 Prata C — Configure um alerta e prove que disparou	29
🥇 Ouro A — O projeto dbt da Aula 3 orquestrado pelo Airflow	30
🥇 Ouro B — Bronze particionada por data (o pré-requisito real de backfill)	30
Troubleshooting — do erro à solução	31
Checklist de saída	32
Apêndice — Plano C: o mesmo lab rodando no seu computador (Astro CLI)	33
C.1 — Pré-requisitos (uma vez)	33
C.2 — Subir o Airflow local	33
C.3 — O resto é igual	33
C.4 — O que muda em relação ao Astro (e é bom saber)	34
O que você acabou de fazer	34





Antes de começar (10 min, em casa ou no início da aula)

Três coisas que, se estiverem prontas, poupam quarenta minutos em sala.

0.1 — Snowflake ativo

Você precisa de um trial do Snowflake ativo. O trial dura 30 dias — se o da Aula 2 expirou, crie outro em signup.snowflake.com (3 minutos). Escolha qualquer nuvem/região; anote o e-mail e a senha.

! MFA — leia antes da Etapa 13

Desde 2026 o Snowflake exige autenticação em dois fatores (MFA) para usuários humanos que entram com senha. Isso pode fazer a conexão do Airflow (Etapa 13) falhar com erro de autenticação/MFA.

Se falhar, não perca tempo tentando de novo: siga a Etapa 13B (usuário de serviço com par de chaves). É o jeito certo de fazer em produção — ferramenta nunca usa senha de pessoa.

Warehouse: contas trial podem nascer SEM o warehouse COMPUTE_WH. A Etapa 13.0 resolve com um comando. Sem ele, a Bronze falha com o erro 000606: No active warehouse selected (aconteceu na validação de 08/10/2026).

0.2 — Conta no GitHub e fork do repositório

O código está em github.com/rafsp/MBA_Aula2708. Faça um FORK para a sua conta (botão "Fork" no canto superior direito). Você vai conectar o SEU fork ao Astro — assim consegue editar a DAG nos desafios Prata sem depender de ninguém.

✓ Verificação

Você consegue abrir github.com/SEU-USUARIO/MBA_Aula2708 e ver as pastas dags/, datasets/ e os arquivos Dockerfile, requirements.txt, packages.txt.

0.3 — Regras dos trials

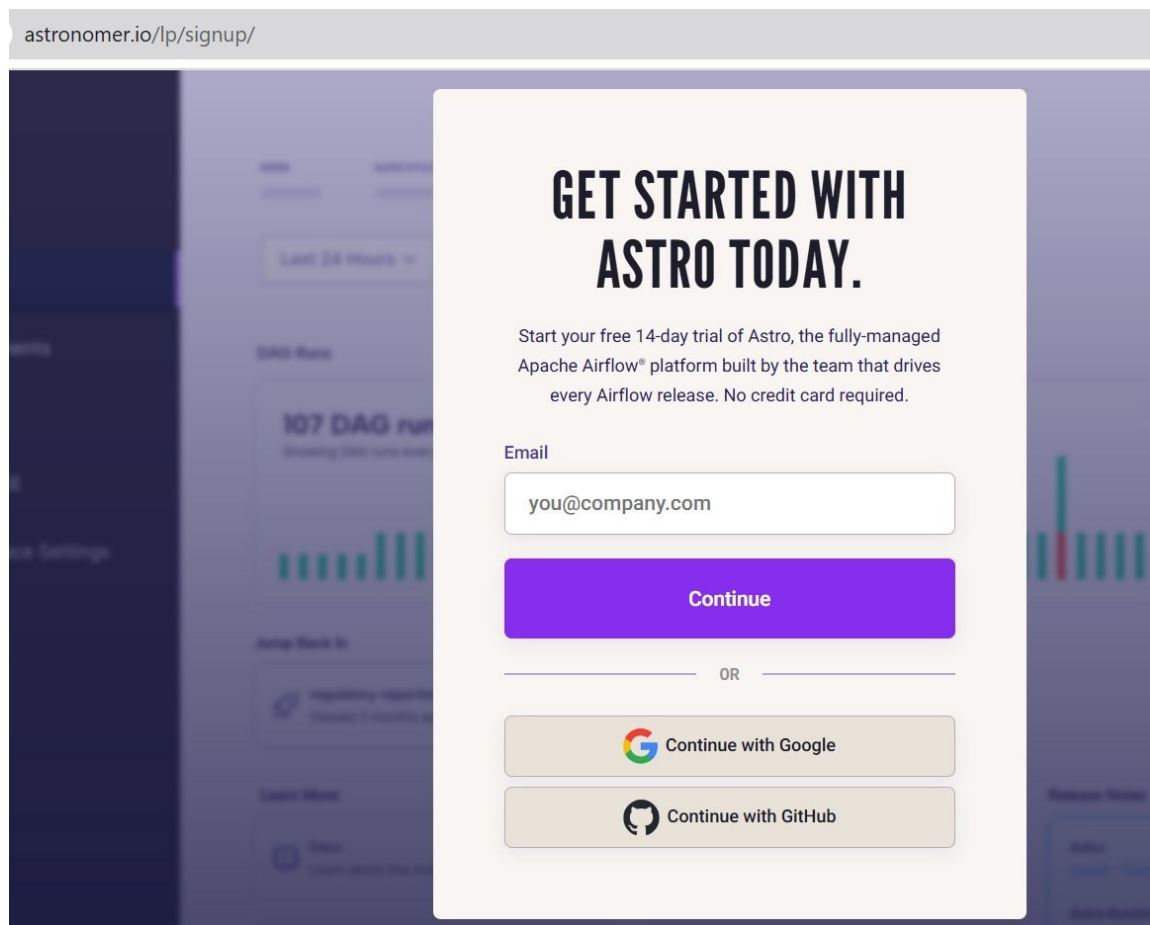
- Astro: 14 dias, US\$ 20 em créditos, sem cartão (valores vistos na tela de signup em ago/2026 — vale o que a tela mostrar). Crie a conta NO DIA da aula, para ela estar viva na entrega dos desafios.
- Snowflake: 30 dias. Anote o Account Identifier no formato ORG-CONTA (ex.: ABCDEFG-XY12345).
- Quando terminar tudo, hiberne ou apague o Deployment do Astro: os créditos são finitos.

Parte 1 — O pipeline rodando (Etapas 1 a 16 · 60–90 min)

Ao final desta parte você terá um pipeline orquestrado pelo Airflow que baixa 7 CSVs do GitHub, carrega no Snowflake, transforma em Bronze/Silver/Gold e valida a qualidade — sem ninguém tocar no Snowflake. Os screenshots são de uma execução real.

Etapa 1 — Criar conta no Astro

Acesse astronomer.io/lp/signup/ no navegador. Preencha seu e-mail ou use "Continue with Google" / "Continue with GitHub".



astronomer.io/lp/signup/

GET STARTED WITH ASTRO TODAY.

Start your free 14-day trial of Astro, the fully-managed Apache Airflow® platform built by the team that drives every Airflow release. No credit card required.

Email

Continue

OR

 Continue with Google

 Continue with GitHub

O trial é gratuito por 14 dias, sem cartão de crédito, com US\$ 20 em créditos.

Etapa 2 — Selecionar o uso

Na tela "Help us tailor your experience", selecione Professional.



ASTRONOMER

Help us tailor your experience

How do you want to use Astro?

 Professional Build, run, and scale data pipelines	 Personal Experiment with personal projects
--	--

Etapa 3 — Configurar Organization e Workspace

Preencha Organization como "Fiap" e Workspace como "Fiap" (ou o nome que preferir). Clique em Continue.



ASTRONOMER

Set up your organization

Add your company and team workspace

Organization *

FIAP

The highest management level in Astro, that can contain multiple Workspaces.

Workspace *

FIAP

Workspaces contain Airflow Deployments, where permitted users can manage and run their Dags.

Continue

Etapa 4 — Pular o template inicial

Na tela "How would you like to begin?", clique em "Or skip this and go to your workspace" no final da página. Vamos configurar o projeto manualmente.



ASTRONOMER

How would you like to begin?

Let's create your first project



Upload my own Dags

Upload your local Dag directory as a .zip file



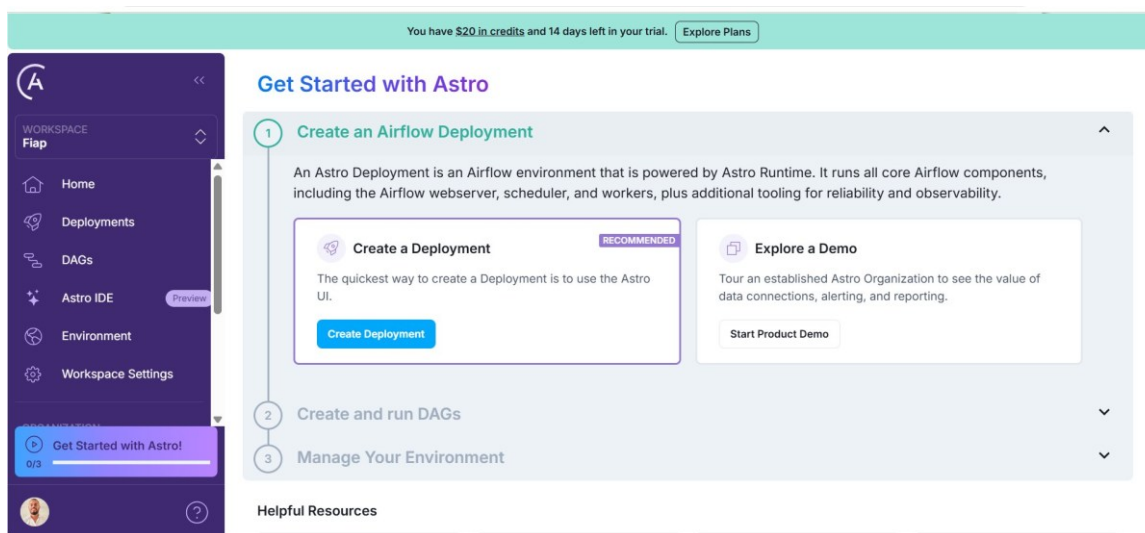
Start with a template

We'll add examples based on the use case you choose

Or skip this and go to your workspace

Etapa 5 — Tela principal do Astro

Você chega na tela principal. No passo 1, clique no botão verde "Create Deployment".



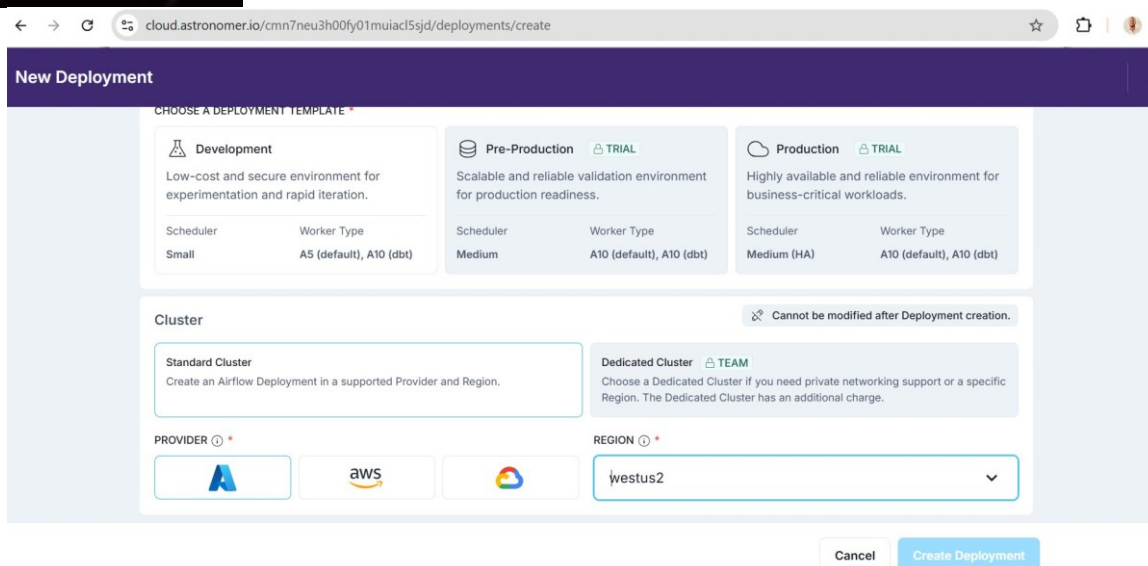
✓ Verificação

No topo aparece "\$20 in credits and 14 days left in your trial" (ou o valor atual do trial).

Etapa 6 — Configurar o Deployment

6.1 — Nome e template

Nome: pipeline-olist. Template: Development (o mais barato). Provider: Astronomer. Cloud: Google Cloud, Region: us-central1.

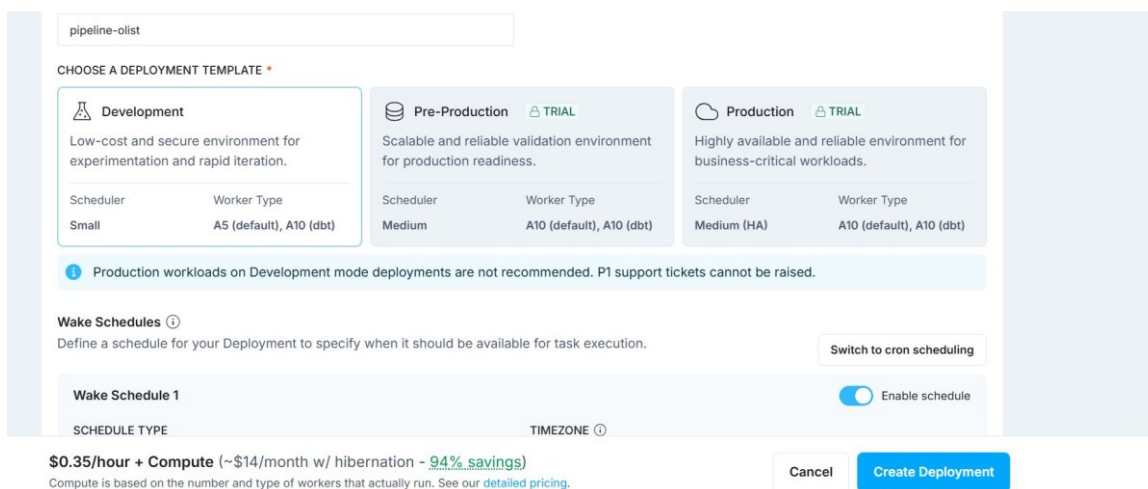


! Por que Google Cloud us-central1

Na preparação desta aula (ago/2026), Deployments criados em Azure (eastus2) e AWS (us-east-1) ficaram presos em CREATING por horas; em Google Cloud us-central1 subiu em minutos. O screenshot acima ainda mostra westus2 — ignore a região dele e escolha us-central1.

6.2 — Template Development selecionado

Confirme Development selecionado. Scheduler: Small. Worker Type: A5. Custo indicado: US\$ 0,35/hora + compute.



6.3 — Wake Schedule

Configure o Wake Schedule para cobrir o horário em que você vai trabalhar. Schedule Type: Custom. Timezone: Brasilia Time (GMT-03:00). Selecione todos os dias e ajuste FROM/TO. Clique em "Create Deployment".



Wake Schedule 1

Enable schedule

SCHEDULE TYPE

Custom

TIMEZONE

(GMT-03:00) Brasília Time - São Paulo

WAKE ON

Mon Tue Wed Thu Fri Sat Sun

FROM

12:30PM

TO

8:00AM (19h 30m)

Add Schedule

Cluster

Cannot be modified after Deployment creation.

Standard Cluster

Create an Airflow Deployment in a supported Provider and Region.

Dedicated Cluster

TEAM

Choose a Dedicated Cluster if you need private networking support or a specific Region. The Dedicated Cluster has an additional charge.

\$0.35/hour + Compute (~\$191.10/month w/ hibernation - 19% savings)

Compute is based on the number and type of workers that actually run. See our [detailed pricing](#).

Cancel

Create Deployment

! Erro comum

Deployment "HIBERNATING" no meio do desafio: o Wake Schedule não cobre o horário.
Deployment → Details → Wake Schedule → ajuste. Ele volta em 2–3 min.

Etapa 7 — Aguardar o Deployment ficar HEALTHY

Aguarde 2–5 minutos. O status muda de "CREATING" para "HEALTHY" e aparece o botão "Open Airflow" no canto superior direito. Repare no campo Docker Image: o Deployment nasce apontando para o Astro Runtime mais novo (em out/2026, 3.3-9, baseado no Airflow 3.3.2). Anote essa versão — ela importa na Etapa 11.

Deployments > pipeline-olist ☆ DEV

CREATING

CLUSTER us-central1

WORKSPACE FIAP

BRANCH MAPPING None - Configure

DOCKER IMAGE (target: 3.3-9)

DAG BUNDLE VERSION

UPDATED a few seconds ago by rafael.novopereira@portoservico.com.br

CREATED a few seconds ago by rafael.novopereira@portoservico.com.br

Overview

Analytics

Logs

Environment

Bundles

Access

Alerts

Incident History

AI Agents

Details

Deploy History

Search Deployments...

1 Deploy Refreshed a few seconds ago

STATUS	TYPE	BUNDLE	TIME	DESCRIPTION	ASTRO RUNTIME	DEPLOYED BY
CREATING	image	—	Today at 3:43 PM	Base Image Deploy	3.3-9 (Based on Airflow 3.3.2)	rafael.novopereira@portoservico.com.br

1-1 of 1 Deploy

Deployments > pipeline-olist ☆ DEV

HEALTHY

CLUSTER us-central1

WORKSPACE FIAP

BRANCH MAPPING None - Configure

DOCKER IMAGE 3.3-9

DAG BUNDLE VERSION

UPDATED 3 minutes ago by rafael.novopereira@portoservico.com.br

CREATED 3 minutes ago by rafael.novopereira@portoservico.com.br

Overview

Analytics

Logs

Environment

Bundles

Access

Alerts

Incident History

AI Agents

Details

Deploy History

Search Deployments...

1 Deploy Refreshed a few seconds ago

STATUS	TYPE	BUNDLE	TIME	DESCRIPTION	ASTRO RUNTIME	DEPLOYED BY
HEALTHY	image	—	Today at 3:43 PM	Base Image Deploy	3.3-9 (Based on Airflow 3.3.2)	rafael.novopereira@portoservico.com.br

1-1 of 1 Deploy

! Dois estados que enganam

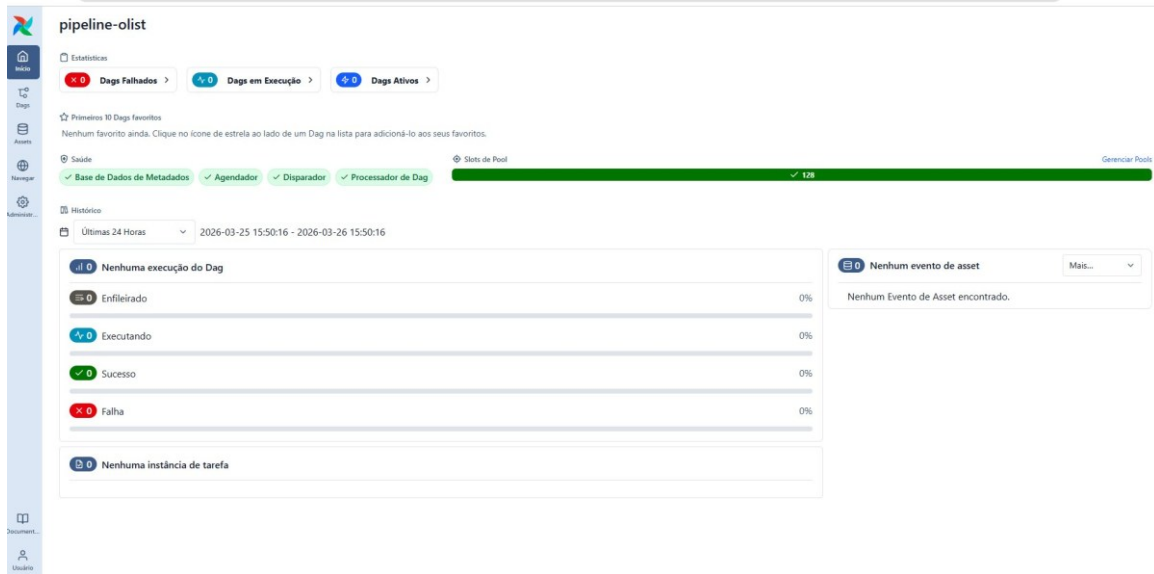
FIAP MBA Data Engineering · Prof. Rafael S Novo Pereira · página 9

CREATING por mais de 15 minutos: não espere. Apague e crie de novo em OUTRA nuvem (Google Cloud us-central1). Três tentativas na mesma nuvem não ajudam.

HIBERNATING logo após criar: o Deployment está pronto, só dormiu porque o Wake Schedule não cobre o horário de agora. Clique em Wake / Override hibernation e ajuste o schedule.

Etapa 8 — Verificar o Airflow

Clique em "Open Airflow". A UI do Airflow 3 abre mostrando os componentes saudáveis: Base de Dados de Metadados, Agendador (Scheduler), Disparador (Triggerer) e Processador de Dag. Todos verdes.



! "Not authorized" ao dar F5 ou colar a URL

A sessão que o botão Open Airflow cria expira. Se aparecer "Not authorized", volte ao Astro e clique em Open Airflow de novo — nunca use favorito nem URL colada. Se persistir, janela anônima + Open Airflow.

💡 Conecte com a aula

São as peças do slide "Arquitetura do Airflow 3": Metadata DB (o caderno), Scheduler (o chefe), Triggerer (quem segura tasks que esperam evento) e DAG Processor (quem lê a receita). Os Workers não aparecem aqui porque nascem como pods só quando uma task roda.

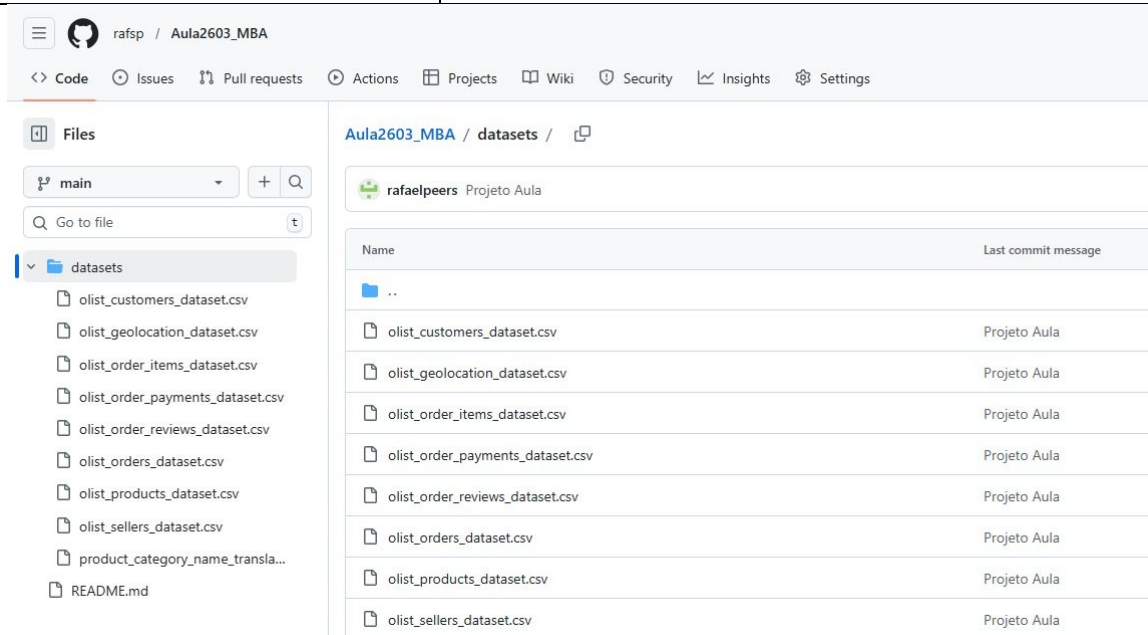
Neste ponto o Airflow está rodando, mas ainda sem nenhuma DAG. Vamos resolver isso nas próximas etapas.

Etapa 9 — O código no GitHub

Abra o seu fork (github.com/SEU-USUARIO/MBA_Aula2708). O repositório contém:

Caminho	O que é
datasets/	9 CSVs do Olist (a DAG usa 7 — geolocation e a tradução de categorias ficam de fora)
dags/pipeline_olist.py	A DAG principal: pipeline_olist_completo (6 tasks)
dags/demo_backfill.py	A demo de backfill vista na aula (Etapa 17)
dags/pipeline_olist_dbt.py	Desafio Ouro: o projeto dbt da Aula 3 orquestrado pelo Airflow (Cosmos)
dags/dbt/olist/	O projeto dbt da Aula 3

Caminho	O que é
setup/01_snowflake_service_user.sql	Plano B da Etapa 13 (usuário de serviço + chave)
Dockerfile · requirements.txt · packages.txt	Receita da imagem Docker: Astro Runtime 3.3-9 (Airflow 3.3.2), provider do Snowflake, pandas, Cosmos, venv do dbt

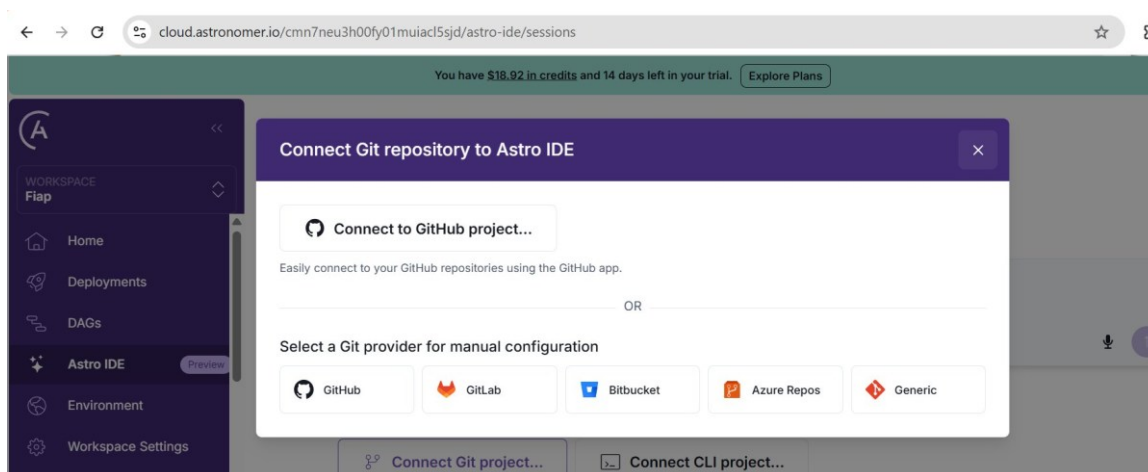


Os três arquivos da raiz são o que o Astro usa para construir a imagem Docker com as dependências certas. Sem eles, a task de Bronze falharia com "No module named snowflake".

Etapa 10 — Conectar o GitHub ao Astro IDE

10.1 — Abrir o Astro IDE

No Astro, clique em "Astro IDE" no menu lateral. Depois em "Connect Git project..."



10.2 — Autorizar o GitHub

Clique em "Connect to GitHub project..." e autorize o acesso ao seu GitHub quando solicitado.

10.3 — Selecionar o repositório



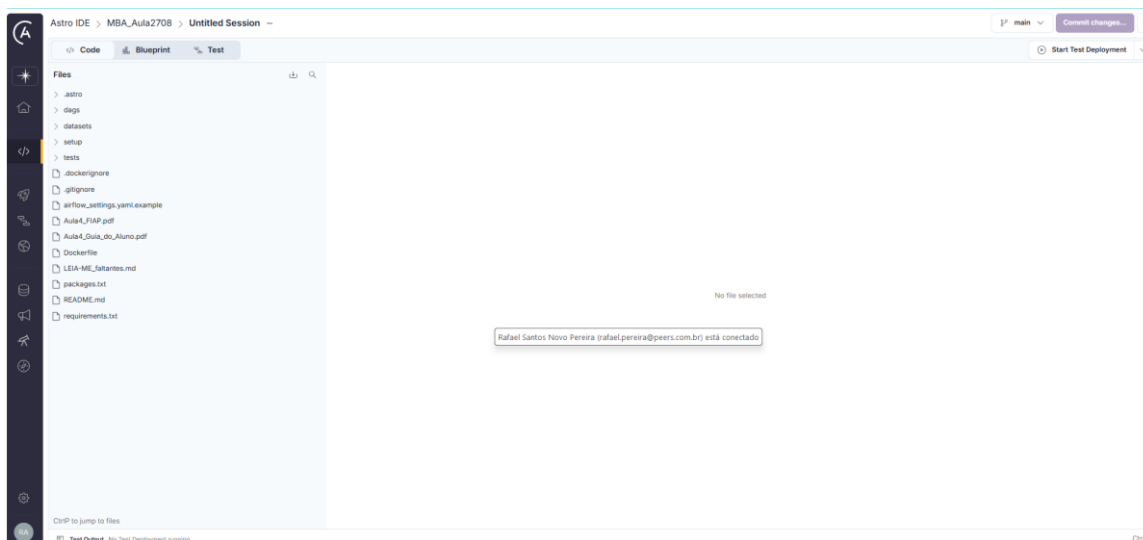
Selecione o SEU fork (MBA_Aula2708). Deixe o campo "Astro Project Path" VAZIO — o projeto está na raiz. Git Branch: main. Clique em Connect.

! Erro comum nº 1 do lab

Preencher "Astro Project Path" com "dags" ou "/". Tem que ficar vazio. Se errou: desconecte o projeto no IDE e conecte de novo.

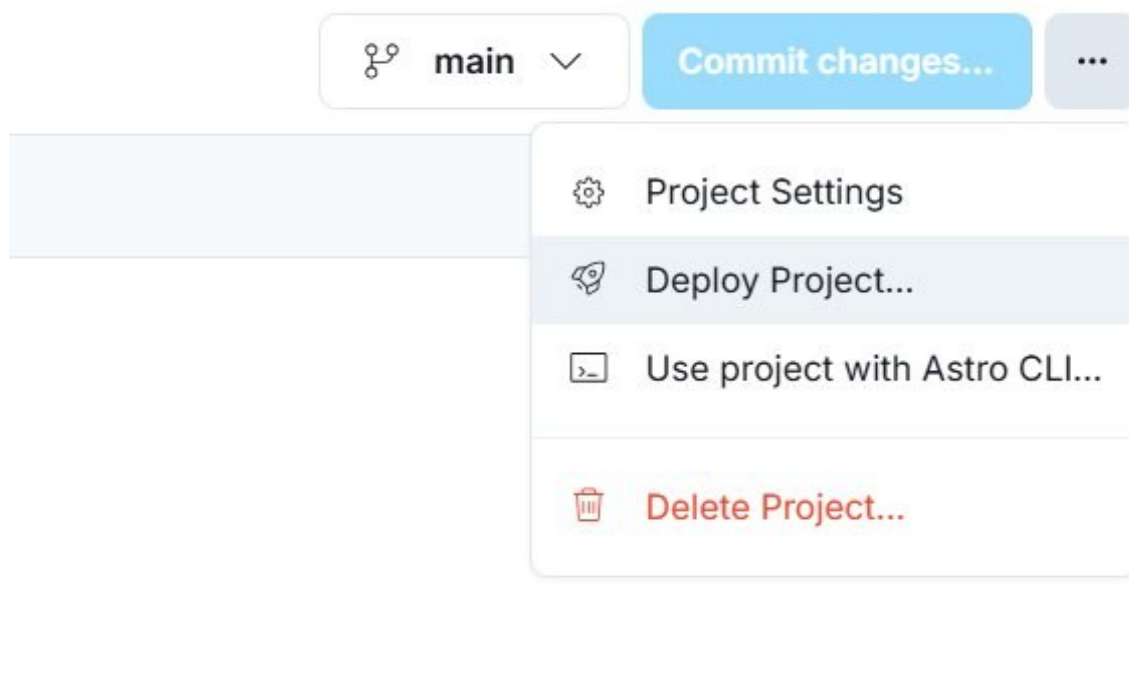
10.4 — IDE conectado

O Astro IDE abre mostrando os arquivos: dags, datasets, setup, Dockerfile, packages.txt, README.md, requirements.txt.

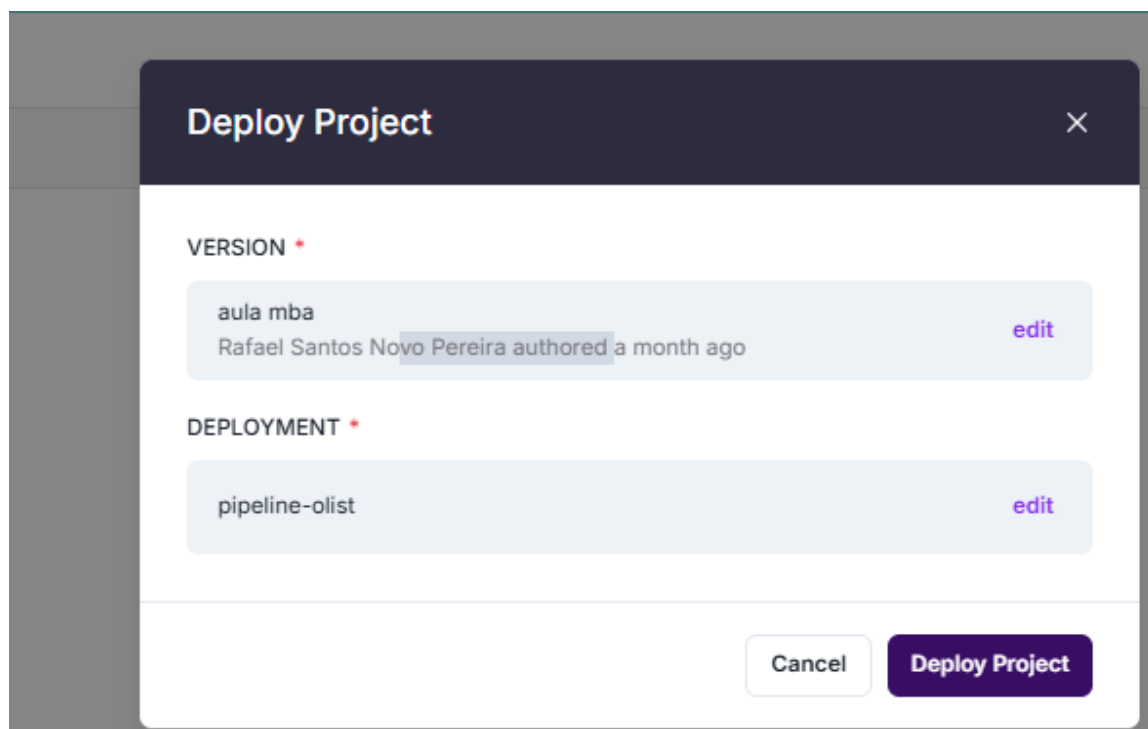


Etapa 11 — Deploy do projeto

Clique nos três pontinhos "..." no canto superior direito do IDE e selecione "Deploy Project..."



Na janela de confirmação, verifique que o Deployment é "pipeline-olist" e clique em "Deploy Project".



Aguarde 5–10 minutos. O Astro clona o repositório, constrói a imagem Docker com as dependências e sobe as DAGs (na validação de 08/10/2026: clone 33 s, build 276 s, upload 308 s). O deploy bom termina com as três fases verdes:



Deployments > pipeline-olist > Today at 4:05 PM

[26d8be2](#) ajuste de versão

...

DEPLOYED	DOCKER IMAGE deploy-2026-10-08T19-05-14	DAG BUNDLE VERSION 2026-10-08T19:10:57.1538788Z	ASTRO RUNTIME 3.3-9 (Based on Airflow 3.3.2)	DEPLOYED 6 minutes ago by GitHub App
----------	--	--	---	---

Deploy log

✓	Clone repository — Completed in 33 seconds	View Logs ▾
✓	Build and push image — Completed in 276 seconds	View Logs ▾
✓	Upload Dags — Completed in 308 seconds	

Bundle snapshot

TYPE	BUNDLE	VERSION	
Dag	main	Today at 4:10 PM	CHANGED BY THIS DEPLOY

! Erro esperado nº 1 — "Image Astro Runtime version X is less than current deployment version Y"

O Deployment nunca aceita uma imagem de runtime MENOR que o dele. Se o Dockerfile do seu fork estiver numa versão antiga (ex.: 3.3-6) e o Deployment nasceu em 3.3-9, o deploy falha com essa mensagem — clone e build passam; a recusa é na hora de aplicar a imagem.

Correção: no IDE, abra o Dockerfile, troque a tag pela versão que o Deployment mostra (ex.: quay.io/astromer/astro-runtime:3.3-9), Commit changes e Deploy Project de novo. O repositório oficial da turma já está em 3.3-9; isso pega forks antigos ou um runtime mais novo lançado depois desta aula.

Deployments > pipeline-olist > Today at 3:51 PM

[007b509](#) aula mba

...

FAILED	DOCKER IMAGE deploy-2026-10-08T18-51-19	DAG BUNDLE VERSION —	ASTRO RUNTIME —	DEPLOYED 6 minutes ago by GitHub App
--------	--	-------------------------	--------------------	---

! Image Astro Runtime version 3.3-6 is less than current deployment Astro Runtime version 3.3-9 (type: CoreError, retryable: true)

Deploy log

✓	Clone repository — Completed in 30 seconds	View Logs ▾
✓	Build and push image — Completed in 281 seconds	View Logs ▾
✓	Upload Dags — Completed in 11 seconds	

Bundle snapshot

No bundles were on the deployment as of this deploy.

💡 O que aconteceu por baixo

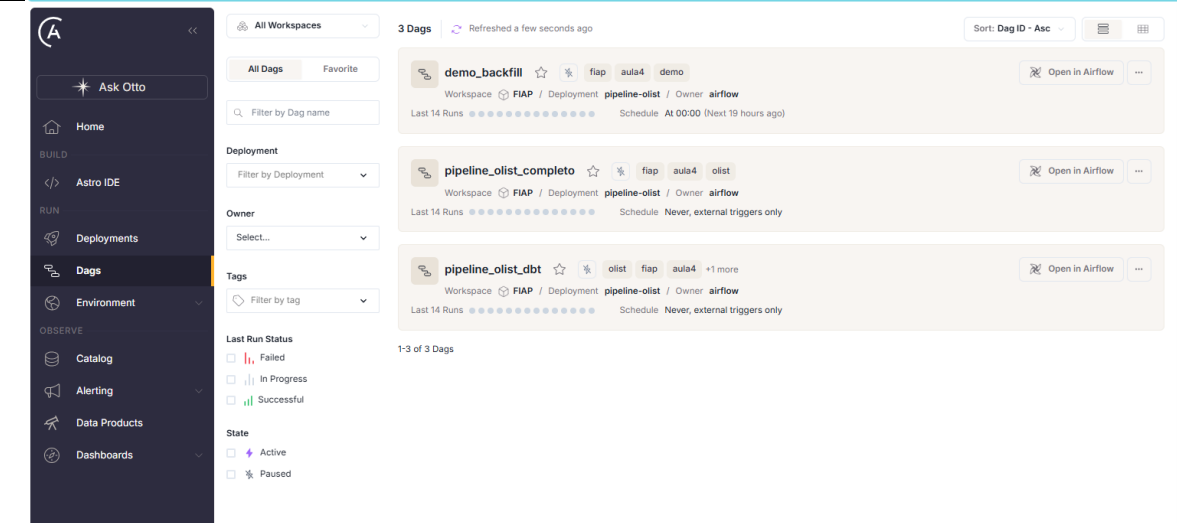
Seu código virou uma imagem Docker (Python + bibliotecas + DAGs). A imagem subiu no cluster Kubernetes do Astro. A partir de agora, cada task que rodar vai nascer como um pod feito dessa imagem. É o slide "Astro: vocês já estão usando Kubernetes".

Etapa 12 — Verificar as DAGs no Airflow



Clique em "Open Airflow". Três DAGs aparecem na lista:

DAG	Para quê	Quando usar
pipeline_olist_completo	O pipeline do lab: inicio → download_and_load_bronze → transform_silver → build_gold → quality_checks → fim	Etapa 14
demo_backfill	A demo da aula: schedule diário + task que lê logical_date	Etapa 17
pipeline_olist_dbt	Desafio Ouro (Cosmos + dbt). NÃO rode agora.	Parte 3



✓ Verificação

As três DAGs estão na lista e nenhuma mostra "Import Error". Se aparecer erro de import, veja a tabela de troubleshooting no fim do guia.

! DAG não aparece?

Espera o Deployment voltar a HEALTHY (3–5 min) e recarregue a página. O DAG Processor só lê os arquivos depois que a imagem nova está no ar.

Etapa 13 — Configurar a Connection com Snowflake

🛑 CHECKPOINT 1 — levante a mão antes de clicar em Salvar

Esta é a etapa mais importante. Sem a Connection correta, a task de Bronze fica vermelha (três tentativas) e as seguintes ficam upstream_failed. Confira os quatro campos com o professor antes de salvar.

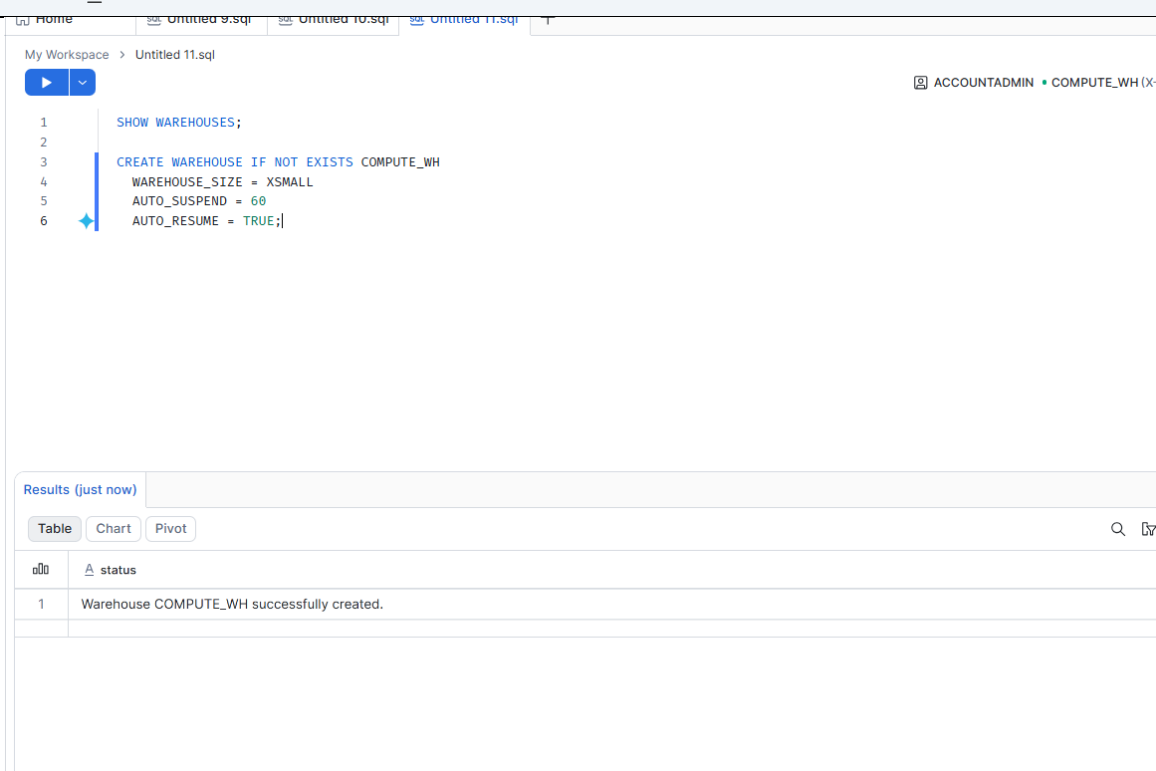
A senha do Snowflake NUNCA fica no código. Ela fica numa Connection do Airflow, criptografada no Metadata DB. O código só diz "use a conexão snowflake_default".

13.0 — Garantir o warehouse (1 comando, evita o erro mais novo do lab)

No Snowflake (Projects → SQL File), rode como ACCOUNTADMIN:

```
SHOW WAREHOUSES;

CREATE WAREHOUSE IF NOT EXISTS COMPUTE_WH
  WAREHOUSE_SIZE = XSMALL
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE;
```



The screenshot shows the Snowflake SQL interface. The top bar indicates the user is ACCOUNTADMIN. The main area displays the SQL commands entered: SHOW WAREHOUSES; and CREATE WAREHOUSE IF NOT EXISTS COMPUTE_WH with parameters WAREHOUSE_SIZE = XSMALL, AUTO_SUSPEND = 60, and AUTO_RESUME = TRUE;. Below the code, the 'Results (just now)' section shows a single result: 'Warehouse COMPUTE_WH successfully created.' The interface includes a sidebar with 'My Workspace' and 'Untitled 11.sql', and a bottom bar with 'Table', 'Chart', and 'Pivot' options.

! Por que isso existe

Descoberta da validação de 08/10/2026: o trial novo do Snowflake veio sem o COMPUTE_WH e a Bronze falhou com 000606: No active warehouse selected — mesmo com a autenticação OK (CREATE/USE DATABASE não precisam de warehouse; o primeiro comando que precisa é o write_pandas). Um comando resolve; depois é só 🧹 Limpar na task.

13.1 — Encontrar suas credenciais do Snowflake

Abra o Snowflake no browser. Clique no seu nome (canto inferior esquerdo) → Account Details. Anote o Account Identifier (formato ORG-CONTA) e o Login name.

Account Details		
Account	Config File	Connectors/Drivers
SQL Commands		
NAME	VALUE	
Account identifier ⓘ	AJQWLVN-XJC53275	
Data sharing account identifier ⓘ	AJQWLVN.XJC53275	
Organization name	AJQWLVN	
Account name	XJC53275	
Account/Server URL	AJQWLVN-XJC53275.snowflakecomputing.com	
Login name ⓘ	RAFAELNOVO	
Role	ACCOUNTADMIN	
Account locator	DHC22010	
Cloud platform	AWS	
Edition	Enterprise	
Learn more		Close

13.2 — Criar a Connection no Airflow

No Airflow, clique na engrenagem (Admin) → Conexões (Connections) → botão "+". Preencha:

Campo	Valor
ID da Conexão (Connection Id)	snowflake_default
Tipo de Conexão (Connection Type)	Snowflake

13.3 — Campos padrão (Etapa 13A — com senha)

Campo	Valor
login	o LOGIN_NAME do seu usuário no Snowflake (veja abaixo — pode ser diferente do nome que aparece no menu)
password	sua senha do Snowflake
schema	PUBLIC

! Erro comum nº 3 do lab — NAME não é LOGIN_NAME

O nome que aparece no menu do Snowflake (ex.: MARIASILVA) é o NAME do usuário. O conector precisa do LOGIN_NAME, que em trial criado por e-mail costuma ser o e-mail. Confira no Snowflake (Projects → SQL File): DESC USER MARIASILVA; e copie o valor da linha LOGIN_NAME para o campo login. Sintoma quando está errado: a Bronze falha em 3 segundos com "390100 ... Incorrect username or password".

Adicionar Conexão

ID da Conexão *

snowflake_default

Tipo de Conexão *

Snowflake

Tipo de conexão faltando? Certifique-se de ter instalado o pacote do provider correspondente ao Airflow.

Campos Padrão

Descrição

login

RAFAELNOVO

password

MbaEngenharia@2016

schema

PUBLIC

Campos Extra

Campos Extra JSON

Salvar

13.4 — Campos Extra JSON

Clique em "Campos Extra JSON" para expandir. Cole o JSON abaixo, trocando o account pelo seu Account Identifier:

```
{"account": "SEU-ORG-SUA-CONTA", "warehouse": "COMPUTE_WH",
  "database": "OLIST_LAB", "role": "ACCOUNTADMIN"}
```

Adicionar Conexão

ID da Conexão *

snowflake_default

Tipo de Conexão *

Snowflake

Tipo de conexão faltando? Certifique-se de ter instalado o pacote do provider correspondente ao Airflow.

Campos Padrão

Campos Extra

Campos Extra JSON

```

1  {
2    "account": "TCIUXVY-MZ32872",
3    "warehouse": "COMPUTE_WH",
4    "database": "OLIST LAB",
5    "role": "ACCOUNTADMIN"
6  }

```

Salvar

! Erro comum nº 2 do lab

account no formato errado. É ORG-CONTA (ex.: ABCDEFG-XY12345). NÃO inclua ".snowflakecomputing.com", "https://" nem a região. Se a URL da sua conta é https://abcdefg-xy12345.snowflakecomputing.com, o account é abcdefg-xy12345.

Clique em Salvar. Depois clique em "Test" (se disponível no seu Deployment) — ou vá direto para a Etapa 14: a task de Bronze é o teste real.

✓ Verificação

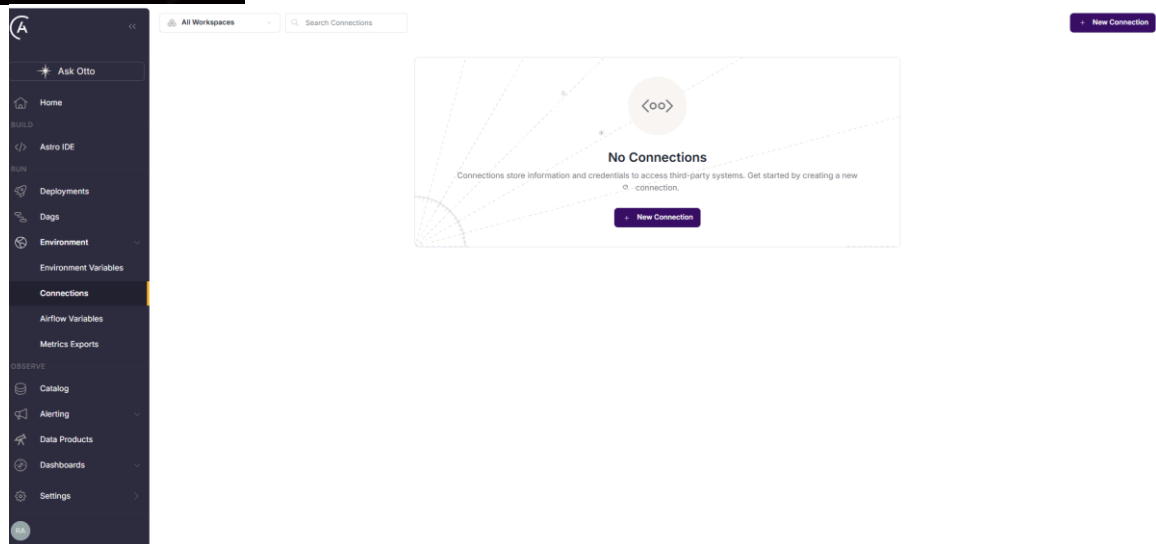
A Connection snowflake_default aparece na lista com tipo Snowflake.

💡 Caiu na tela de Connections do ASTRO em vez do Airflow? Dá para usar, mas tem três pegadinhas

O Astro agora tem a própria tela de conexões (Environment → Connections), com formulários prontos de Snowflake. Se preferir esse caminho:

- 1) O Connection ID NÃO pode ser editado depois de criado e PRECISA ser snowflake_default. Criou como "conexao" ou "snowflake"? Apague e crie de novo.
- 2) Warehouse, database e role ficam escondidos em "More options". Sem eles, aparece o erro 000606.
- 3) A conexão só chega ao Deployment com Auto-linking ALL DEPLOYMENTS = On (ou vínculo manual).

Em sala, recomendamos o caminho do Airflow (13.2): é um formulário só e o ID já vai certo.



connections

New Snowflake Connection

×

WORKSPACE *

FIAP

×

▼

Workspaces where you have create permission

CONNECTION ID *

conexao

Airflow Connection ID. Must be unique across all connections.

USERNAME *

rafsp

The username to use for authenticating against Snowflake.

ACCOUNT LOCATOR *

GP12345

The unique identifier of the Snowflake account to connect to.

PASSWORD *

my_password

The password to use for authenticating against Snowflake.

DESCRIPTION

e.g., Read-only Snowflake connection for analytics

Add helpful context about this connection, such as permissions or usage notes.

> More options

Auto-linking

When "On", all current and future resources in your Workspace will have access to this connection. When "Off", you can choose which resources have access to this connection.

ALL DEPLOYMENTS

Off

ALL ASTRO IDE PROJECTS

Off

13B — Plano B: usuário de serviço com par de chaves (se a Bronze falhar com erro de MFA/autenticação)

Se, na Etapa 14, o log da task `download_and_load_bronze` mostrar algo como "Multi-factor authentication is required", "MFA" ou "250001" (com a senha certa), o Snowflake está exigindo MFA do seu usuário humano. A solução — e a prática correta em produção — é um usuário de serviço que se autentica por chave, não por senha. Leva 10 minutos.

Passo 1 — Gerar o par de chaves no seu computador. Abra um terminal (macOS/Linux: Terminal; Windows: Git Bash ou PowerShell com OpenSSL) e rode:

```
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt
openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub
```

Isso cria `rsa_key.p8` (chave PRIVADA — nunca compartilhe, nunca faça commit) e `rsa_key.pub` (chave pública).

Passo 2 — Registrar a chave pública no Snowflake. Abra `rsa_key.pub` num editor. Copie SOMENTE o miolo (as linhas entre `-----BEGIN PUBLIC KEY-----` e `-----END PUBLIC KEY-----`), junte tudo numa linha. No Snowflake, em Projects → SQL File, abra o arquivo `setup/01_snowflake_service_user.sql` do repositório, cole o miolo no campo `RSA_PUBLIC_KEY` e execute o script inteiro como `ACCOUNTADMIN`.

O script cria: a role `AIRFLOW_ROLE` (com o mínimo necessário sobre `OLIST_LAB` e `COMPUTE_WH`), o usuário `AIRFLOW_SVC` com `TYPE = SERVICE` (não humano → fora da regra de MFA) e registra a chave. No fim, `DESC USER AIRFLOW_SVC` deve mostrar `RSA_PUBLIC_KEY_FP` preenchido.

Passo 3 — Refazer a Connection no Airflow. Edite `snowflake_default` (ou apague e crie de novo):

Campo	Valor
login	<code>AIRFLOW_SVC</code>
password	(deixe VAZIO)
schema	<code>PUBLIC</code>

Extra JSON — cole o CONTEÚDO INTEIRO do `rsa_key.p8` (com as linhas `BEGIN/END`) no campo `private_key_content`, trocando as quebras de linha por `\n`:

```
{
  "account": "SEU-ORG-SUA-CONTA",
  "warehouse": "COMPUTE_WH",
  "database": "OLIST_LAB",
  "role": "AIRFLOW_ROLE",
  "private_key_content": "-----BEGIN PRIVATE KEY-----\nMIIEvQIBADANBg...\n...\n-----END PRIVATE KEY-----"}

```

💡 Dica para montar o JSON sem erro

No terminal: `python3 -c "import json;print(json.dumps(open('rsa_key.p8').read()))"` — isso imprime a chave já escapada com `\n`, pronta para colar depois de `"private_key_content":` .

✅ Verificação

Rode a Etapa 14 de novo. Se a Bronze ficar verde, a Connection por chave está funcionando. Para o desafio Ouro (dbt), adicione a variável de ambiente `SNOWFLAKE_AUTH=keypair` no Deployment (Astro → Deployment → Environment) — a DAG `dbt` usa isso para saber que deve autenticar por chave.

Etapa 14 — Executar o pipeline

Volte para a lista de DAGs. Clique em `pipeline_olist_completo`. Ligue o toggle ao lado do nome (a DAG nasce pausada). Clique em "Acionar" (o botão play no canto superior direito). Selecione "Execução Única" e confirme. Acione UMA vez e espere: a DAG tem `max_active_runs=1` — um segundo clique só cria uma execução na fila.

Acionar Dag - pipeline_olist_completo

Execução Única

Acionar uma única execução deste Dag

Backfill

Executar este Dag para um intervalo de datas

Opções Avançadas

☒ Despausar pipeline_olist_completo ao acionar

▶ Acionar

Acompanhe as tasks na lateral: elas mudam de cor conforme avançam pela jornada (scheduled → queued → running → success). A execução completa leva cerca de 2 minutos — a Bronze é a mais longa (baixa 50 MB e grava 7 tabelas).

pipeline-olist

ajuste de versão · Astro Git Integration · 47 minutos ago

pipeline_olist_completo

manual_2026-10-08T19:52:31.012953+00:00

00:00:03.279

00:00:01.637

Inicio

download_and_load_bronze

transform_silver

build_gold

quality_checks

fim

manual_2026-10-08T19:52:31.012953+00:00

Executando

Data Lógica

2026-10-08 16:52:29

Tipo de Execução

Manual

Data Inicial

2026-10-08 16:52:31

Data Final

Duração

00:00:00.351

Nome do Usuário que Disparou

cmuzvt4jalwox01r18kty0a47

Versão(s) do Dag

v1

Adicionar uma nota...

Instâncias de Tarefa

Asset Events

Log de Auditoria

Código

Detalhes

Resource metrics

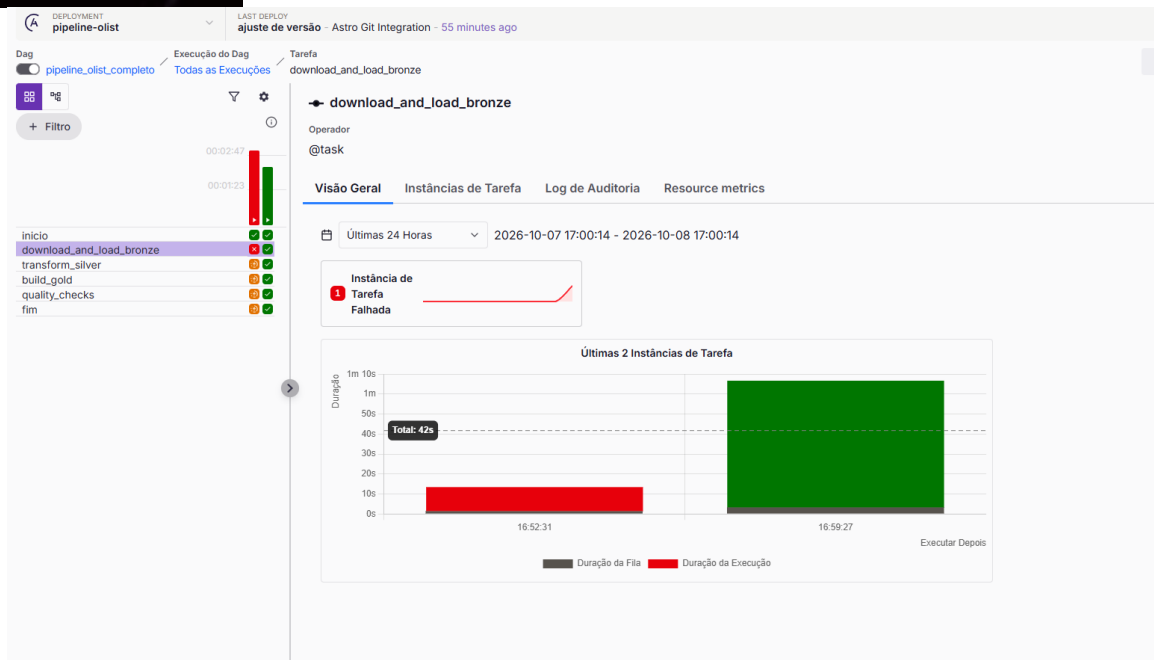
6 Instâncias de Tarefa

ID da Tarefa	Índice do Mapa	Estado	Data Inicial	Número de Tentativas	Operador	Duração
quality_checks		Sem Status		0	@task	
build_gold		Sem Status		0	@task	
transform_silver		Sem Status		0	@task	
download_and_load_bronze		Executando	2026-10-08 16:52:34	1	@task	
fim		Sem Status		0	EmptyOperator	
inicio		Sucesso	2026-10-08 16:52:31	1	EmptyOperator	

! Bronze vermelha ou laranja ("Pronto para tentar novamente")?

Falhou em poucos segundos = erro de conexão; o retry não vai resolver. Clique na task → Logs → leia a ÚLTIMA linha. "390100 ... Incorrect username or password" → login (LOGIN_NAME) ou senha errados (13.3). "MFA", "250001" → Etapa 13B. "conn_id snowflake_default isn't defined" → a Connection não foi salva. "could not connect" / "404" → account no Extra JSON (13.4). "No module named" → Project Path preenchido (10.3). "000606 No active warehouse" → Etapa 13.0. Depois de corrigir: na linha da task, o ícone ☹ (Limpar/Clear) → ela e as seguintes rodam de novo na hora, sem esperar o retry.

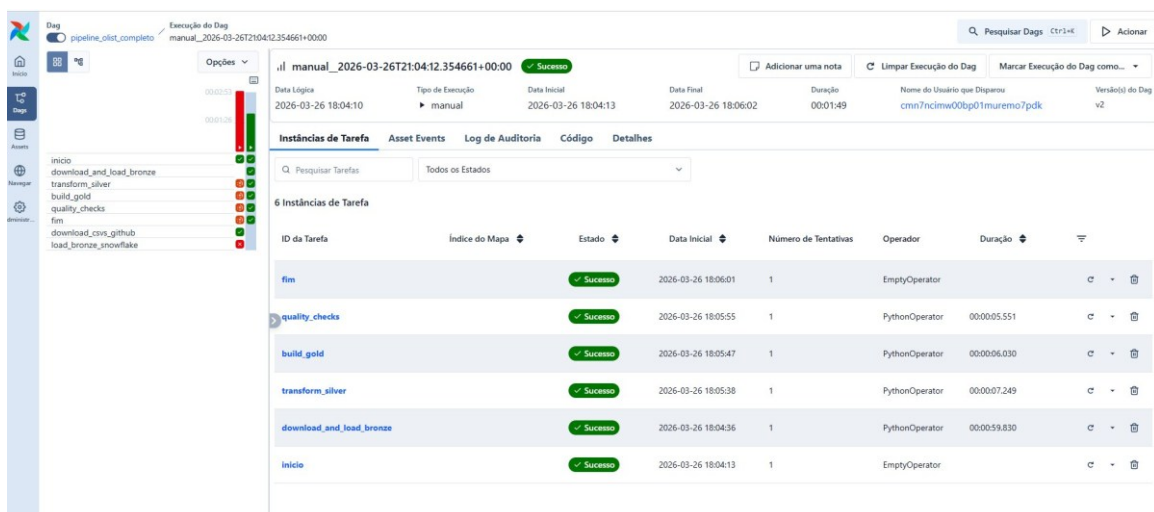
FIAP MBA Data Engineering · Prof. Rafael S Novo Pereira · página 23



Antes e depois do Clear, na validação de 08/10/2026: a tentativa das 16:52 falhou (faltava o warehouse da Etapa 13.0); criado o COMPUTE_WH, o Clear reexecutou às 16:59 e ficou verde. Errar, ler o log e corrigir É o fluxo de trabalho.

Etapa 15 — Pipeline completo com sucesso

Quando todas as tasks ficarem verdes, o pipeline terminou. Veja a duração de cada task e o total.



Observe: início → download_and_load_bronze (59 s) → transform_silver (7 s) → build_gold (6 s) → quality_checks (5 s) → fim. Total ≈ 1 min 49 s. Abra o log de quality_checks: ele imprime as 4 regras com OK e o top 5 de estados por receita.

✓ CHECKPOINT 2 — evidências do desafio Bronze

Tire agora os 3 screenshots (com seu nome de usuário visível no canto da tela): (1) a Grid com as 6 tasks verdes; (2) o log de download_and_load_bronze mostrando as 7 tabelas e o número de linhas; (3) — na Etapa 16 — o resultado do SELECT na Gold.

Etapa 16 — Verificar os resultados no Snowflake

Abra o Snowflake em Projects → SQL File (a antiga "Worksheet") e execute:

```
SELECT * FROM OLIST_LAB.PUBLIC.GOLD_RECEITA_ESTADO ORDER BY RECEITA_TOTAL DESC
LIMIT 5;
```

1  | SELECT * FROM OLIST_LAB.PUBLIC.GOLD_RECEITA_ESTADO LIMIT 5;

2

 Add to Chat |  Explain |  Quick edit |  Format

Results (just now)

Table | Chart

5 rows 86ms

	ESTADO	# TOTAL_PEDIDOS	# RECEITA_TOTAL	# TICKET_MEDIO	# DIAS_ENTREGA_MEDIO	# PCT_ATRASO	CT_ATUALIZADO_EM
1	SP	41746	5998226.96	143.69	8.7	5.72	2026-03-26 14:05:50.714 -0700
2	RJ	12852	2144379.69	166.85	15.2	12.95	2026-03-26 14:05:50.714 -0700
3	MG	11635	1872257.26	160.92	11.9	5.48	2026-03-26 14:05:50.714 -0700
4	RS	5466	890898.54	162.99	15.2	6.99	2026-03-26 14:05:50.714 -0700
5	PR	5045	811156.38	160.78	11.9	4.88	2026-03-26 14:05:50.714 -0700

Compare com os valores esperados — eles são reproduzíveis, quem rodar o pipeline vê os mesmos:

ESTADO	TOTAL_PEDIDOS	RECEITA_TOTAL	TICKET_MEDIO	DIAS_ENTREGA_MEDIO	PCT_ATRASO
SP	41746	5998226.96	143.69	8.7	5.72
RJ	12852	2144379.69	166.85	15.2	12.95
MG	11635	1872257.26	160.92	11.9	5.48

Agora as outras tabelas que o pipeline criou:

```
SHOW TABLES IN OLIST_LAB.PUBLIC; -- 7 BRONZE_*, SILVER_PEDIDOS, 2
GOLD_*
SELECT * FROM OLIST_LAB.PUBLIC.GOLD_VENDAS_MENSAL ORDER BY MES;
SELECT COUNT(*) FROM OLIST_LAB.PUBLIC.SILVER_PEDIDOS; -- 99441 (a Silver tem 1
linha por pedido)
```



Search for anything

Recently viewed

- PUBLIC
- TASK_BRONZE
- BRONZE_VENDAS
- INFORMATION_SCHEMA
- TASK_GOLD

Data products

No data product listings available

Catalog explorer

QY05139

Find objects by name

- Data Products
- Databases (7)
 - LAB_PIPELINE
 - OLIST_LAB (2)
 - INFORMATION_SCHEMA
 - PUBLIC (10)
 - Tables (10)
 - BRONZE_CUSTOMERS
 - BRONZE_ORDER_ITEMS
 - BRONZE_ORDERS
 - BRONZE_PAYMENTS
 - BRONZE_PRODUCTS
 - BRONZE_REVIEWS
 - BRONZE_SELLERS
 - GOLD_RECEITA_ESTADO
 - GOLD_VENDAS_MENSAL
 - SILVER_PEDIDOS
 - PC_DBT_DB
 - SNOWFLAKE
 - SNOWFLAKE_LEARNING_DB
 - SNOWFLAKE_SAMPLE_DATA
 - USERSRAFP



O Catalog depois da execução validada de 08/10/2026: as 10 tabelas do lab em OLIST_LAB.PUBLIC.

☒ **Desafio Bronze concluído**

SP lidera com ~R\$ 6 M em 41.746 pedidos; RJ tem 12,95% de atraso. Esses dados saíram do GitHub, viraram Bronze, Silver e Gold e passaram por 4 validações — sem ninguém tocar no Snowflake. Você usou Kubernetes sem perceber: cada task rodou num pod isolado.



Parte 2 — Duas experiências que valem mais do que parecem (15 min)

Etapa 17 — Backfill: a task não sabe "hoje"

Você viu o professor fazer isso na aula. Agora é você. Abra a DAG `demo_backfill`. Ela tem 12 linhas e três diferenças em relação à principal: `schedule="@daily"`, `start_date` no passado, e a task lê `logical_date`.

1. Ligue o toggle da DAG. Em segundos aparece sozinha UMA execução, com a data de hoje (é o `@daily`: ao despausar, o Airflow cria o intervalo corrente). Deixe terminar.
2. Clique em "Acionar". No diálogo, marque a opção "Backfill" (à direita de "Execução Única").
3. Data inicial: 7 dias atrás. Data final: ontem (deixe hoje de fora — já existe). Marque NÃO reprocessar execuções existentes. Acionar.
4. Em segundos aparecem 7 execuções na Grid (7 colunas). Cada uma termina em menos de 1 minuto.
5. Abra o log de uma execução do meio: "Processando o dia 2026-xx-xx". Abra outra: outra data.

The screenshot shows the Airflow web interface. At the top, there's a banner indicating a backfill is in progress for the 'demo_backfill' DAG, with a date range from 2026-10-01 to 2026-10-07. Below this, a table lists the tasks, showing 'que_dia_eu_sou' as successful. The 'Logs' tab is selected, displaying the log for the 'que_dia_eu_sou' task. The log shows the task's execution details, including the date being processed (2026-10-08) and the success message.

O banner azul de backfill em progresso é novo na UI do Airflow 3.3 (capturado em 08/10/2026, backfill de 01/10 a 07/10).

The screenshot shows the 'Logs' tab for the 'que_dia_eu_sou' task. The log displays the task's execution details, including the date being processed (2026-10-08) and the success message.

💡 O que você acabou de provar

A mesma função Python rodou 8 vezes com 8 datas diferentes — e a data veio do orquestrador, não de `datetime.now()`. Um script que usa "agora" nunca consegue reprocessar o passado. Um pipeline que usa `logical_date` consegue. Essa é a diferença entre script e pipeline — e é o coração do desafio Prata B.

Etapa 18 — Idempotência: rodar duas vezes é seguro?



1. No Snowflake, anote o valor de `_ATUALIZADO_EM` da linha SP em `GOLD_RECEITA_ESTADO`.
2. No Airflow, acione `pipeline_olist_completo` de novo (Execução Única). Espere ficar verde (~2 min).
3. No Snowflake, rode o `SELECT` da Etapa 16 de novo.

Os números são idênticos; só `_ATUALIZADO_EM` mudou. Nenhuma linha duplicada, nenhuma receita dobrada. Por quê? A Bronze grava com `overwrite=True` e Silver/Gold usam `CREATE OR REPLACE`. Abra `dags/pipeline_olist.py` no Astro IDE e encontre essas três palavras.

Por que isso importa

Retry só é seguro em pipeline idempotente. Se a Bronze fizesse `INSERT` sem apagar, cada retry duplicaria 99 mil pedidos — e o dashboard mostraria o dobro da receita. Idempotência primeiro, retry depois. É o pilar nº 1 do checklist da aula.

Parte 3 — Desafios (entrega individual pelo portal)

Entrega: um PDF com os screenshots (seu nome de usuário visível) + o link do seu fork no GitHub. Prazo: [DEFINIR].

Nível	O que entregar	Vale
 Bronze — obrigatório	Os 3 screenshots do Checkpoint 2 (Grid verde, log da Bronze com as 7 tabelas, SELECT na Gold)	até 7,0
 Prata — escolha 1 ou 2	Código no seu fork + evidência de execução (screenshots) + explicação de 5 linhas	+1,5 cada
 Ouro — bônus	DAG verde + explicação de 5 linhas do que mudou	+1,0 (teto 10)

Prata A — Faça um quality check falhar de verdade

Objetivo: entender que o quality check é um contrato e que a DAG protege o dashboard quando ele é violado.

1. No Astro IDE, abra dags/pipeline_olist.py e encontre a lista regras dentro de quality_checks.

```
("Nenhum estado com atraso > 10%", f"SELECT COUNT(*) FROM {gold} WHERE PCT_ATRASO > 10", lambda v: v == 0),
```

2. Commit + Deploy Project. Acione a DAG. quality_checks fica vermelha (3 tentativas — retry não resolve regra de negócio) e fim fica upstream_failed.
3. Abra o log de quality_checks: ele mostra FAIL na regra nova e o motivo.

Evidências

Screenshot da Grid com quality_checks vermelha e fim upstream_failed; screenshot do log com a linha FAIL; 5 linhas explicando: por que o fim não rodou, por que o retry não ajudou, e o que um time faria com esse alerta (corrigir o dado? relaxar a regra? bloquear o dashboard?).

Prata B — Dê um calendário à DAG e faça backfill

Objetivo: aplicar na DAG principal as três mudanças da demo_backfill.

1. Em dags/pipeline_olist.py: troque schedule=None por schedule="@daily" (start_date já está no passado; catchup=False continua).

```
def download_and_load_bronze(**context)) e imprimir context["logical_date"] na primeira linha do log.
```

2. Commit + Deploy. Acionar ▾ → Backfill → 3 dias (anteontem, ontem, hoje) → não reprocessar existentes.
3. Três execuções aparecem na Grid. Abra o log da Bronze de cada uma: três datas diferentes.

Pergunta para a explicação

As três execuções carregaram exatamente os mesmos dados. Por quê? O que precisaria mudar na Bronze para cada execução carregar SÓ os pedidos daquele dia? (Essa resposta é o desafio Ouro B.)

Prata C — Configure um alerta e prove que disparou

Objetivo: fechar o pilar "Alertas" que ficou com X no checklist da aula. Duas rotas:

Rota 1 — Astro Alerts (sem código). No Astro (não no Airflow): Deployment → Alerts → Add Alert → tipo "DAG failure" (ou "Task failure") → DAG pipeline_olist_completo → canal

e-mail com o seu endereço. Salve. Provoque uma falha (use a regra do Prata A) e receba o e-mail.

Rota 2 — on_failure_callback (com código). Crie um webhook no Slack ou no Discord (Configurações do servidor → Integrações → Webhooks). Em dags/pipeline_olist.py, adicione uma função que faça `requests.post(WEBHOOK_URL, json={"content": f"Falhou: {context['task_instance'].task_id}"})` e registre em `default_args: "on_failure_callback": notificar`. Guarde a URL do webhook numa Variable do Airflow (Admin → Variables), nunca no código.

✓ Evidências

Screenshot do alerta configurado + screenshot do e-mail/mensagem recebido, com a task e o horário.

🏆 Ouro A — O projeto dbt da Aula 3 orquestrado pelo Airflow

A DAG pipeline_olist_dbt já está no repositório. Ela usa o Cosmos para transformar cada modelo dbt em uma task do Airflow (10 tasks, com run e test separados) e grava em schemas próprios: BRONZE, STAGING, SILVER, GOLD.

1. Se a sua Connection é por chave (Etapa 13B): Astro → Deployment → Environment → adicione SNOWFLAKE_AUTH = keypair. Se é por senha, não precisa fazer nada.
2. No Airflow, acione pipeline_olist_dbt. A primeira task carrega a Bronze em OLIST_LAB.BRONZE; o grupo dbt_transform roda os modelos; quality_checks valida OLIST_LAB.GOLD.FCT_RECEITA_ESTADO.
3. Abra a aba Graph: o lineage stg → int → fct aparece como tasks conectadas. Compare com o lineage que você viu no dbt Cloud na Aula 3.

✓ Evidências

Screenshot da Grid/Graph com as 10 tasks verdes; SELECT em OLIST_LAB.GOLD.FCT_RECEITA_ESTADO; 5 linhas comparando: o que muda entre transformar em Python (pipeline principal) e em dbt (esta DAG)? Onde ficam os testes em cada caso?

🏆 Ouro B — Bronze particionada por data (o pré-requisito real de backfill)

Objetivo: fazer cada execução carregar só os pedidos do seu logical_date — o que torna o backfill de verdade útil.

1. Na Bronze, depois de ler olist_orders_dataset.csv, filtre o DataFrame pela data de compra igual ao logical_date (o dataset vai de 2016 a 2018 — use `start_date=datetime(2017, 1, 1)` e escolha datas dessa faixa no backfill).
2. Troque `overwrite=True` por: `DELETE FROM BRONZE_ORDERS WHERE DATE(ORDER_PURCHASE_TIMESTAMP) = '<dia>'` seguido de `append` (`overwrite=False`). Isso é idempotência POR DATA.
3. Backfill de 3 dias de 2017. Confira no Snowflake: `SELECT DATE(ORDER_PURCHASE_TIMESTAMP), COUNT(*) FROM BRONZE_ORDERS GROUP BY 1` — só os 3 dias, sem duplicatas mesmo rodando duas vezes.

✓ Evidências

Trecho do código no fork; Grid com as 3 execuções; resultado do GROUP BY antes e depois de rodar o mesmo dia duas vezes (mesma contagem).

Troubleshooting — do erro à solução

Sintoma	Causa provável	Solução
Deploy falha: "Image Astro Runtime version X is less than current deployment version Y"	Dockerfile numa versão menor que a do Deployment	Dockerfile → mesma tag do Deployment (ex.: 3.3-9) → Commit → Deploy Project (Etapa 11).
Bronze vermelha: "000606 No active warehouse selected"	Trial novo do Snowflake sem o COMPUTE_WH (a autenticação passou)	Etapa 13.0: CREATE WAREHOUSE IF NOT EXISTS COMPUTE_WH. Depois ↺ Limpar na task.
Deploy falha: "Deployment is not in a healthy state" + toast "Waking deployment"	Deployment hibernado na hora do deploy	O Astro acorda sozinho. Espere HEALTHY (2–3 min) e Deploy Project de novo.
DAG não aparece na lista após o deploy	Deployment ainda construindo a imagem	Espere HEALTHY (3–5 min) e recarregue. Persistiu? Veja "Import Errors" no topo da lista.
"Import Error" em pipeline_olist_dbt	Cosmos ou dbt não instalados na imagem	Confira requirements.txt (astronomer-cosmos==1.15.1) e o Dockerfile (venv dbt). Redeploy. Não afeta a DAG principal.
Bronze vermelha: "Multi-factor authentication", "MFA", "250001"	Snowflake exigindo MFA do usuário humano	Etapa 13B (usuário de serviço + chave).
Bronze em 3 s: "390100 Incorrect username or password"	login é o NAME, não o LOGIN_NAME (ou senha errada)	DESC USER <nome>; → copie LOGIN_NAME para o campo login. Se entra no browser e não entra aqui com os mesmos dados → MFA → Etapa 13B.
Bronze vermelha: "could not connect", "404", "account"	Account Identifier no formato errado	Extra JSON: "account": "ORG-CONTA", sem .snowflakecomputing.com, sem https.
Bronze vermelha: "No module named snowflake" / "pandas"	Imagem sem dependências	Etapa 10.3: Astro Project Path deve estar VAZIO. Reconecte e redeploy.
Bronze vermelha: "Insufficient privileges" / "does not exist"	Role sem permissão em OLIST_LAB	Com senha: role ACCOUNTADMIN no Extra. Com chave: rode o setup SQL inteiro (grants).
"Not authorized" na tela do Airflow	Sessão do Open Airflow expirou	Astro → Open Airflow de novo. Nunca por URL colada; se persistir, janela anônima.
quality_checks vermelha, Bronze/Silver/Gold verdes	Warehouse suspenso no meio, ou regra do Prata A	Clique na task → Clear. Se foi o Prata A: comportamento esperado.
transform_silver: "Object BRONZE_ORDERS does not exist"	Bronze carregou em outro database/schema	Extra JSON: database OLIST_LAB. A DAG usa nomes qualificados OLIST_LAB.PUBLIC.*.
Deployment CREATING há mais de 15 min	Provisionamento travado naquela nuvem/cluster	Apague e crie em outra nuvem: Google Cloud us-central1 subiu em minutos quando Azure e AWS travaram (ago/2026). Se persistir em todas: suporte pelo chat do Astro, com o Deployment ID.
Deployment HIBERNATING	Fora do Wake Schedule	Wake / Override hibernation e ajuste o Wake Schedule. Volta em 2–3 min.
Task fica "queued" por minutos	Worker subindo (pod) ou Deployment acordando	Normal até ~1 min. Além disso: veja Deployment → Health.
Cliquei "Acionar" duas vezes — duas execuções	max_active_runs=1 segura a segunda na fila	Apague a segunda (lixeira ao lado do nome da execução) e acompanhe só a primeira.
Opção Backfill não aparece no diálogo Acionar	DAG com schedule=None	Backfill exige schedule (Prata B).



Checklist de saída

- ☐ Deployment HEALTHY e Wake Schedule cobrindo o horário em que vou fazer os desafios
- ☐ pipeline_olist_completo com 6 tasks verdes (screenshot)
- ☐ Log de download_and_load_bronze com as 7 tabelas (screenshot)
- ☐ SELECT na GOLD_RECEITA_ESTADO com SP = 41746 pedidos (screenshot)
- ☐ Etapa 17: 8 execuções da demo_backfill com datas diferentes
- ☐ Etapa 18: rodei duas vezes e os números não mudaram
- ☐ Escolhi meu(s) desafio(s) Prata e sei que evidência entregar
- ☐ Sei onde está o log quando algo fica vermelho — e leio a última linha primeiro

Apêndice — Plano C: o mesmo lab rodando no seu computador (Astro CLI)

💡 Quando usar este apêndice

Se o Astro não conseguir criar o Deployment (status CREATING por mais de 15 minutos, em qualquer nuvem) ou se o trial acabar antes da entrega. O mesmo repositório, as mesmas três DAGs e a mesma Connection rodam localmente com o Astro CLI. O que muda: em vez de Kubernetes, as tasks rodam em containers Docker na sua máquina.

C.1 — Pré-requisitos (uma vez)

- Docker Desktop instalado e ABERTO (docker.com/products/docker-desktop). Sem Docker rodando, nada abaixo funciona.
- Astro CLI: macOS → brew install astro · Windows → winget install -e --id Astronomer.Astro · Linux → curl -sSL install.astronomer.io | sudo bash -s
- Confira: astro version (deve responder com a versão do CLI).
- Espaço em disco: a primeira subida baixa a imagem do Astro Runtime (~1,5 GB) e instala o dbt no venv — 5 a 10 minutos na primeira vez, segundos nas seguintes.

C.2 — Subir o Airflow local

1. Clone o SEU fork: git clone https://github.com/SEU-USUARIO/MBA_Aula2708.git e entre na pasta: cd MBA_Aula2708
2. Suba tudo: astro dev start — o CLI constrói a imagem a partir do Dockerfile (com requirements.txt), e inicia postgres, scheduler, api-server, dag-processor e triggerer.
3. Quando aparecer "Airflow is starting up!" com a URL, abra http://localhost:8080 — usuário admin, senha admin.
4. Na lista, as três DAGs: pipeline_olist_completo, demo_backfill e pipeline_olist_dbt.

```
git clone https://github.com/SEU-USUARIO/MBA_Aula2708.git
cd MBA_Aula2708
astro dev start
# http://localhost:8080 · admin / admin
```

! Erro comum

"Cannot connect to the Docker daemon": o Docker Desktop não está aberto. Abra, espere o ícone ficar estável e rode astro dev start de novo. Porta 8080 ocupada: astro config set webserver.port 8081 e suba de novo.

C.3 — O resto é igual

A partir daqui, siga o guia normalmente: Etapa 13 (Connection snowflake_default — mesmos campos, mesmo Extra JSON, mesmo plano B), Etapa 14 (Trigger), Etapa 16 (SELECT no Snowflake), Etapa 17 (Backfill) e Etapa 18 (idempotência). Os desafios Prata e Ouro também funcionam localmente. Comandos úteis:

Comando	Para quê
astro dev restart	Depois de editar Dockerfile ou requirements.txt (reconstrói a imagem). Connections e histórico são mantidos.
astro dev stop	Para os containers sem apagar nada. astro dev start volta de onde parou.
astro dev kill	Apaga TUDO, inclusive Connections e histórico. Só se quiser começar do zero.

Comando	Para quê
<code>astro dev logs -f</code>	Logs do scheduler/api-server em tempo real (útil para Import Error).
<code>astro dev parse</code>	Confere se as DAGs carregam sem erro, sem subir a UI.

C.4 — O que muda em relação ao Astro (e é bom saber)

No Astro (nuvem)	Local (astro dev start)
Cada task roda num pod Kubernetes isolado — /tmp de uma task não existe para a próxima	Tasks rodam no mesmo container do scheduler — /tmp É compartilhado. A versão com download e load separados que quebrou no Astro funcionaria aqui. Por isso a DAG mantém tudo numa task: código que só funciona no seu computador não é código de produção.
Escala workers sozinho; paga por hora	Um worker; gratuito; limitado pela sua máquina
Deploy por Git → imagem → cluster	Edita o arquivo em dags/ e o DAG Processor recarrega em segundos — bom para os desafios
Astro Alerts na interface	Não existe; Prata C só pela rota 2 (on_failure_callback + webhook)
Wake Schedule / hibernação	Não existe; astro dev stop quando terminar

✓ Evidências dos desafios no plano C

Valem os mesmos screenshots (Grid, logs, SELECT). Como não há nome de usuário do Astro na tela, inclua no PDF um screenshot do terminal com `astro dev ps` mostrando os containers e o caminho da pasta com o seu usuário do sistema.

O que você acabou de fazer

1. Criou um ambiente Airflow 3.3 gerenciado na nuvem (Astro), rodando em Kubernetes.
2. Conectou o seu fork do GitHub e fez deploy do código (Dockerfile + requirements.txt + DAGs em Python).
3. Configurou a conexão com o Snowflake com credenciais seguras — e, se precisou, do jeito que se faz em produção: usuário de serviço com chave.
4. Executou um pipeline que baixou 7 CSVs, criou 7 tabelas Bronze, 1 Silver com JOINS, 2 Gold com métricas de negócio e rodou 4 validações de qualidade.
5. Fez backfill de 7 dias e provou que a task usa a data do orquestrador, não "agora".
6. Rodou o pipeline duas vezes e provou que ele é idempotente.

Na vida real, a única diferença é: a fonte seria uma API ou um blob storage em vez do GitHub, o schedule seria diário em vez de manual, e o alerta iria para o Slack. O conceito é exatamente o mesmo — e você acabou de operá-lo.